# Go-Back-N and Selective Repeat over a Satellite Link

Computer Network-Assignment 2-Task 3-4

## 1. Problem and Approach

A disaster-response Emergency Operations Centre (EOC) exchanges casualty lists, supply requests and rescue assignments with field teams over a satellite link (R = 2 Mbps, RTT = 500 ms, L = 1,000 bytes = 8,000 bits). The link loses, corrupts and delays both packets and ACKs, so every packet must arrive correctly and in order. Because the bandwidth-delay product is large, a stop-and-wait protocol (rdt 3.0) would use the link very poorly; pipelining is required. I therefore built a simulated unreliable channel and implemented two sliding-window protocols on top of it, following the Chapter 3 design: Go-Back-N (GBN) and Selective Repeat (SR). rdt 3.0 provides the foundation (checksum, ACKs, timer, retransmission), extended to windows.

**Assumptions.** Transmission time is L/R = 4 ms per packet; one-way propagation delay is RTT/2 = 250 ms. ACKs take no transmission time and suffer the same loss, corruption and delay as data. A delayed packet gets an extra delay uniform in 0–200 ms. The retransmission timeout is 1,000 ms, which exceeds the worst-case RTT of about 900 ms (a premature timeout combined with wrapped sequence numbers produced a wrong delivery in one test at 600 ms). Sequence numbers use k = 3 bits (modulo 8), and the random seed is 1044 (last four digits of my roll number).

## 2. Implementation

The program is an event-driven simulator in Python written from scratch, with no reliable-transport library. A priority queue of timed events (packet arrival, ACK arrival, timeout) drives a simulated clock, so runs are deterministic for a given seed. Each packet carries a type, sequence number, payload and a 16-bit Internet checksum. The channel randomly drops a packet, flips a bit in the payload (which the receiver detects as a checksum mismatch and discards), or adds extra delay. A log line is printed for every send, receive, ACK, loss, corruption, delay, timeout and retransmission. All parameters (protocol, packets, N, k, loss, corruption, delay, timeout) are configurable.

**GBN** (gbn.py) uses the slide variables base and nextseqnum at the sender and expectedseqnum at the receiver. The receiver accepts only the next in-order packet and sends cumulative ACKs; out-of-order and corrupted packets are discarded and the last in-order ACK is repeated. The sender keeps one timer for the oldest unACKed packet; on timeout it resends every outstanding packet from base to nextseqnum−1. A cumulative ACK slides base forward.

**SR** (sr.py) keeps a sender window [base, base+N−1] with a timer per packet and individual ACKs, and a receiver window starting at rcv_base. The receiver buffers out-of-order packets inside its window, ACKs each one, and delivers data in order as rcv_base advances; old duplicates are re-ACKed. On timeout only the missing packet is resent. The code enforces $N \le 2^k - 1$ for GBN and $N \le 2^{(k-1)}$ for SR so that wrapped sequence numbers cannot be confused.

## 3. Experiments

Both protocols transferred 200 packets with N = 4 and k = 3 under three conditions: A) a reliable channel (no loss or corruption); B) packet loss of 0.10; C) mixed errors with loss 0.05, corruption 0.05 and delay probability 0.10. The values are hardcoded in the programs. In every run all 200 packets were delivered correctly and in order (verified by comparing the delivered data with the original).

## 4. Results

| Scenario | Protocol | Transmitted | Retransmissions | Delivered | Execution time (s) | Throughput (Mbps) |
|---|---|---|---|---|---|---|
| A: Reliable | GBN | 200 | 0 | 200 | 25.21 | 0.0635 |
| A: Reliable | SR | 200 | 0 | 200 | 25.21 | 0.0635 |
| B: Loss 0.10 | GBN | 313 | 113 | 200 | 60.92 | 0.0263 |
| B: Loss 0.10 | SR | 257 | 57 | 200 | 58.94 | 0.0271 |
| C: Mixed | GBN | 296 | 96 | 200 | 57.45 | 0.0278 |
| C: Mixed | SR | 242 | 42 | 200 | 56.63 | 0.0283 |

*Table 1: GBN vs SR results (200 packets, N = 4, k = 3, timeout 1,000 ms, seed 1044). Throughput = delivered bits / execution time.*

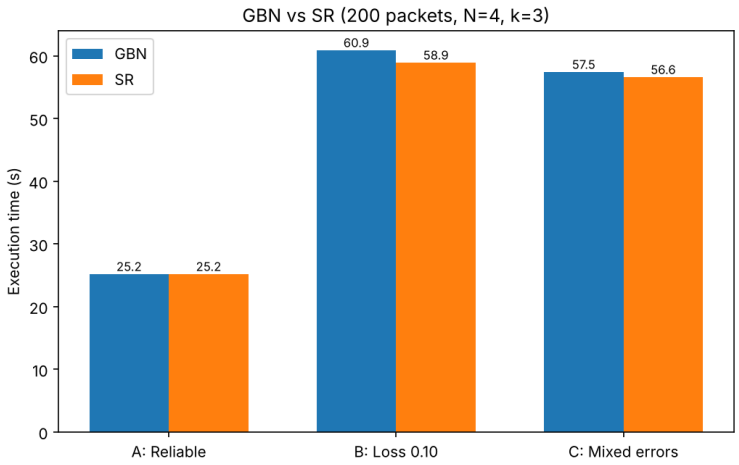



*Figure 1: Execution time of GBN and SR in each scenario.*

## 5. Comparison

On a reliable channel GBN and SR are identical (200 packets, 25.21 s), since no packet is ever retransmitted. Once errors appear, GBN must resend the whole outstanding window after a timeout and its receiver discards out-of-order packets, so it needs far more retransmissions than SR: 113 vs 57 in scenario B and 96 vs 42 in scenario C. SR resends only the packet that timed out and buffers out-of-order packets, so less bandwidth is wasted and it finishes slightly sooner (58.94 s vs 60.92 s in B; 56.63 s vs 57.45 s in C). The time difference is small because with N = 4 and a 500 ms RTT the window is far smaller than the bandwidth-delay product, so both protocols spend most of the transfer waiting for ACKs or timers rather than transmitting. The cost of SR is extra complexity: a timer per packet and receiver buffering.

## 6. Conclusion

Both protocols were implemented from scratch and delivered all data correctly and in order under loss, corruption and delay. SR is more efficient than GBN on a lossy satellite link, with about half the retransmissions and a shorter transfer time, while GBN is simpler to implement. For the EOC network, SR is the better choice, and a larger window together with a timeout set just above the RTT would raise throughput much further, since throughput is currently limited by the small window rather than the link rate.